项目名称：走起（RouteGo）

一句话定位：

从“被安排路线”到“主动设计行程”的旅行工具

核心理念：

帮助用户掌控自由行程中的节奏感、空间感与时间感

产品说明：

RouteGo 是一个地图驱动的旅行路线设计工具。用户可以在地图上自由添加地点，调整顺序、选择交通方式，并通过时间轴统一管理整条行程。

与传统导航或攻略不同，本产品强调“用户主动构建路线”，而不是被动接受推荐路

目录

项目名称：走起（RouteGo）..... 1

产品背景与问题定义..... 2

产品思路丨从“问题结构”到“产品结构”.....3

核心功能丨路线编辑与时间系统设计.....5

竞品对比丨产品取舍与定位.....9

核心问题探索丨“附近逛逛”为什么难做..... 12

用户反馈丨真实使用与迭代.....17

技术实现丨关键问题与解决路径..... 19

项目总结丨从功能到结构的认知转变.....29

产品背景与问题定义

在自由行规划中，用户面临的核心问题不是信息不足，而是行程缺乏结构与掌控感。现有工具虽然丰富，但无法让用户从“拼凑行程”转变为“设计行程结构”。

这一问题本质上是“行程结构不可控”，可以拆解为四个层面：

1. 路径不可控

   传统地图工具以“起点-终点”为核心，缺乏对多点行程的整体控制能力。用户很难在一个统一视角下调整整条路线顺序，只能反复输入与试错。

2. 信息碎片化

   用户需要在多个工具之间频繁切换，例如：在地图中查看位置，在攻略平台查找景点，在备忘录中记录行程。这种跨工具操作不仅效率低，也使行程始终处于割裂状态，难以形成整体。

3. 空间不可控

   用户通常只知道“要去哪些地方”，但难以理解这些地点在城市中的相对位置关系。景点之间的方位、距离与分布缺乏直观感知，行程更像是一组孤立的点，而不是一条有结构的路径。在这种情况下，用户很难建立对整条行程的空间认知，行程更像是被拆散的信息集合，而不是一条连续的路径。

4. 时间不可控

   大多数工具只提供单段路径时间，缺乏对整条行程的时间统筹能力。用户无法清晰判断一天的安排是否合理，也难以及时发现时间冲突或节奏问题。

因此，用户缺的并不是“更多信息”，而是：

一个让用户能够主动设计并掌控整条行程结构的工具

产品思路丨从“问题结构”到“产品结构”

在明确“行程结构不可控”的问题后，我并没有直接拆功能，而是先对“行程”进行结构化抽象：行程本质上不是一组地点，而是一个由空间顺序与时间节奏组成的结构系统。

1. 路径结构（空间顺序）：决定“去哪里、按什么顺序去”，是整条行程的骨架。
2. 空间结构（位置关系）：决定各个地点在城市中的分布关系，帮助用户建立整体空间认知，而不是孤立地看每一个点。
3. 时间结构（节奏安排）：决定每一段行程的时间分配，包括停留时间、路程时间与整体节奏。

在此基础上，我形成了三个核心设计原则：

1. 行程必须“可编辑”：用户需要能够直接操作整条路线，包括调整顺序、增删节点，而不是反复输入参数。
2. 行程必须“可解释”：每一段路径都需要有明确的语义（如交通方式、时间消耗），而不是一条黑盒路线。
3. 行程必须“可感知”：用户需要同时理解空间分布与时间节奏，从而建立对整条行程的整体认知。

产品的所有功能，均会围绕“行程结构的构建过程”展开。

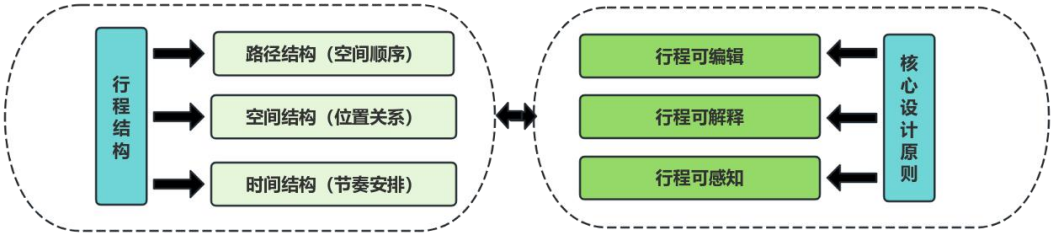


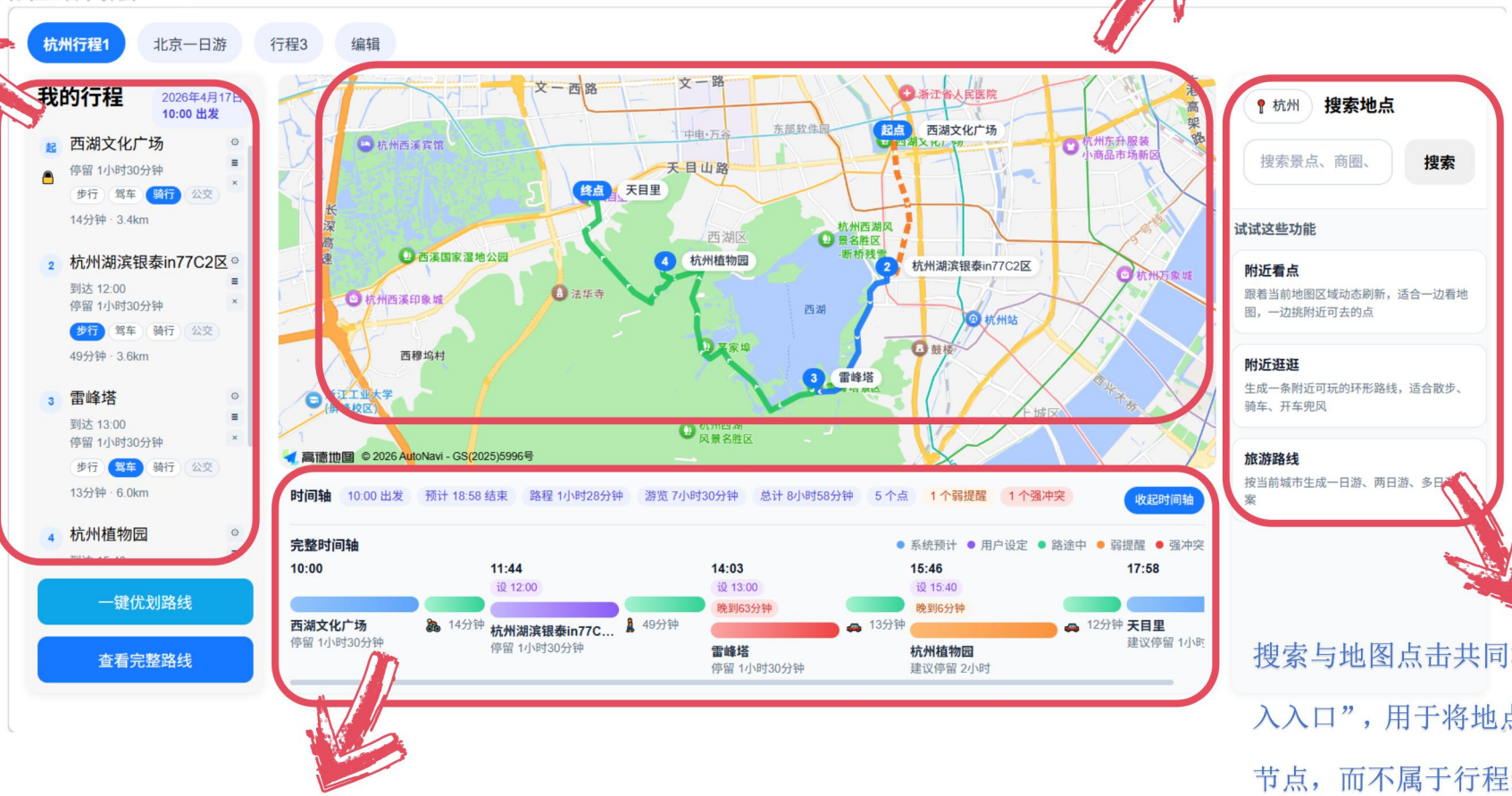
图 1 行程结构与产品设计映射图

左侧行程列表用于组织所有节点的顺序，是整条路线的结构骨架。用户可以通过拖动排序、增删节点，直接调整路径结构，而不是反复重新输入路线参数。

行程列表（路径结构层）

地图不仅用于展示位置，更承担“空间感知”的核心作用，帮助用户理解各个地点在城市中的分布关系。用户通过点击地图或搜索地点，将空间信息转化为行程节点。

地图区域（空间输入层）



搜索与地图点击共同构成“空间输入入口”，用于将地点转化为行程节点，而不属于行程结构本身。

时间系统（结构管理层）

底部时间轴统一整条行程的时间信息，包括出发时间、停留时间、路程时间与整体节奏。通过时间可视化与冲突提示，用户可以判断行程是否合理，并进行动态调整。

核心功能一丨路线编辑能力（结构控制）

在 RouteGo 中，用户可以直接操作的“行程结构”。围绕“路径结构可控”，我重点设计了三类核心编辑能力：

1. 拖动排序：路径结构的直接重构能力

用户可以在行程列表中通过拖动节点，直接调整整条路线的顺序。这一设计将传统“重新输入起终点”的方式，转变为对整条路径结构的直接编辑，使路线调整从“多次试错”变为“即时重排”。

本质：将“路径规划”从参数输入问题，转变为结构操作问题。

2. 起终点锁定：结构稳定与约束机制

用户可以锁定起点或终点，避免在一键优化或拖动过程中被打乱。这一机制解决了用户在“局部调整”与“整体优化”之间的冲突，使系统优化不会破坏用户已有的结构意图。

本质：在自动优化与用户控制之间建立边界。

3. 分段交通方式：路径语义表达能力

用户可以为每一段路径单独选择交通方式（步行/骑行/驾车）。相比默认的单一出行方式，这一设计使每一段路径具备明确语义，使路线从“计算结果”转变为“可理解的结构”。

本质：让路径从“黑盒计算”变为“可解释系统”。

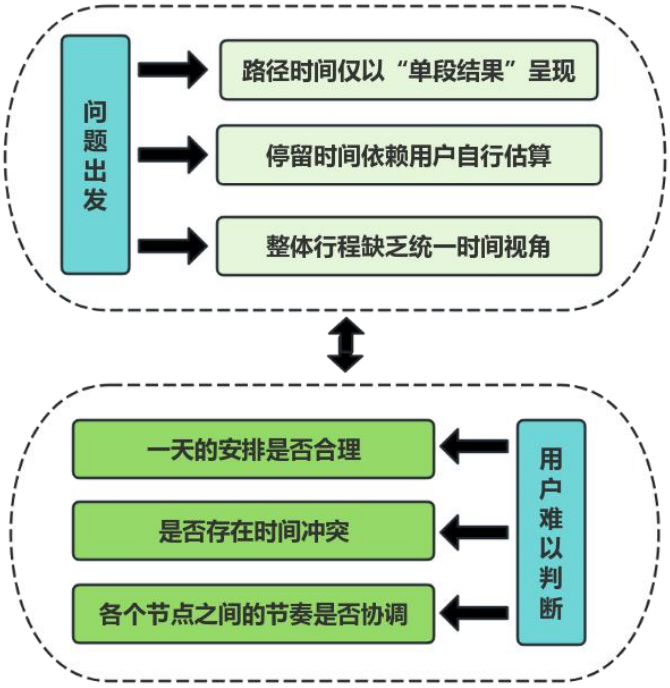
4. 一键优化：结构重排能力

在用户选定多个地点后，系统可以自动对路径顺序进行优化，生成更合理的行程结构。与传统导航不同，该能力不是替代用户决策，而是在用户给定节点集合的基础上，对路径结构进行优化建议。同时，用户可以通过锁定节点来约束优化范围，避免关键节点被调整。

本质：在用户控制基础上提供结构优化能力，实现“人机协同”的路径设计。

核心功能二丨时间系统（节奏控制能力）

在 RouteGo 中，时间并不是路径计算的附属信息，而是与空间结构同等重要的核心维度。 围绕“时间可控”，我构建了一套完整的时间系统，用于统一管理整条行程的节奏。 现有时间处理方式的问题如下结构：



为了解决上述问题，我没有直接叠加功能，而是先对“行程时间”进行结构拆解，将其从单一结果转变为由多个层次共同构成的时间系统：

1. 路程时间（系统计算）：

   基于路径规划结果，自动生成每一段路径的时间消耗。

   作用：提供基础时间数据，反映节点之间的移动成本。

2. 停留时间（用户设定+系统推荐）：

   用户可以为每个节点设置停留时间；在未设定时，系统会提供默认推荐值（如景点建议游览时长），帮助用户快速完成行程构建。

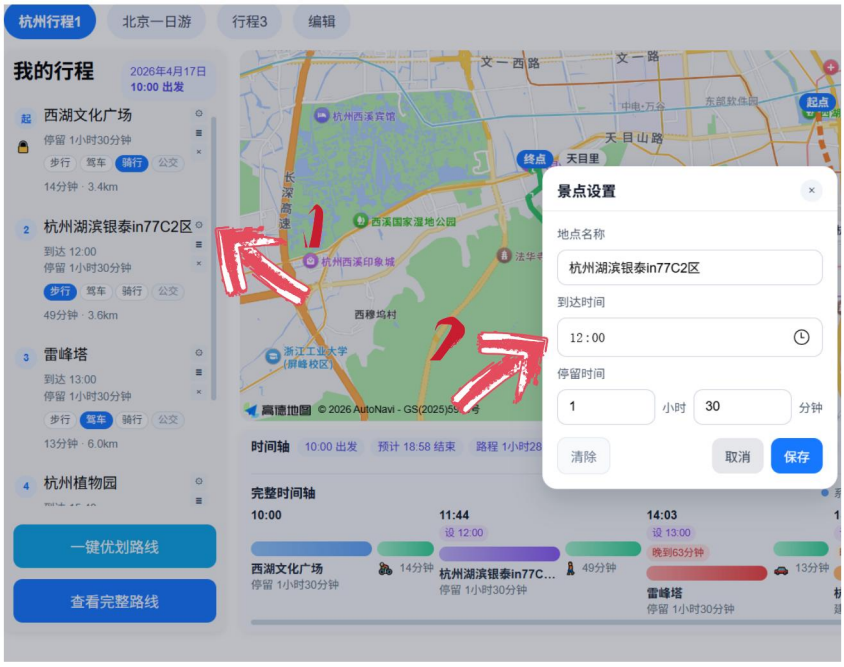
   作用：体现用户意图，并补全行程中“停留行为”的时间信息。

3. 总行程时间（整体节奏）：

系统将“路程时间”与“停留时间”进行整合，形成完整的行程时间结构，并在时间轴中统一展示。

作用：从整体视角反映行程节奏，使用户能够判断安排是否合理。

通过上述拆解，时间不再是路径计算的附属结果，而是由“移动时间”与“停留行为”共同构成的完整结构，为后续的节奏判断提供基础。



在时间结构建立之后，我将整条行程的时间信息统一可视化为一条连续的时间轴。时间轴不仅呈现单一时间点，而是以“区间 + 节点”的形式整合多维信息，包括行程起始时间、各节点的预计到达时间、停留区间以及节点之间的路程区间。

在同一视图中，用户可以直观地看到每一段“移动时间”与“停留时间”的分布关系，以及它们在一天中的整体排列顺序，从而将原本零散的时间信息转化为一条连续的时间结构。

在具体设计上，我围绕“可读性优先”对时间轴进行了多项优化。例如，为避免某些时间区间过短而无法被识别，我为时间轴设置了最小显示长度，使即使是较短的路程或停留时间，也能够在视觉上被感知，而不会被压缩为不可见的片段。

同时，通过分段颜色与区间划分，对不同类型的时间（如停留区间与路程区间）进行区分，并标记关键节点的时间信息，帮助用户快速建立对行程节奏的整体认知。此外，时间轴在展示上并不追求绝对精确的比例还原，而是在可读性与信息表达之间进行平衡，使用户能够在短时间内理解结构，而不需要逐段计算时间。

本质在于：让时间从“计算结果”转变为“可以被理解的结构”，从而降低用户理解与判断行程节奏的成本。

在时间轴的基础上，我进一步引入了基于“用户设定时间”的冲突提示机制，使时间系统从单纯展示结果，升级为对行程可行性的判断工具。系统首先根据用户设定的出发时间与路径结构，生成整条行程的预计到达时间。当用户未进行额外约束时，时间轴仅作为参考节奏存在；而当用户为某一节点设定“必须到达时间”后，系统会将该设定与预计到达时间进行对比，从而判断是否存在偏差。

在偏差判断上，我并未采用简单的“是否冲突”二值逻辑，而是根据偏差程度进行分级处理：

- 预计到达时间早于设定时间：不提示（视为可接受的时间冗余）
- 略晚于设定时间（一定阈值内）：弱提醒（提示轻微延迟风险）
- 明显晚于设定时间（超过阈值）：强提醒（高亮提示无法按时到达）

通过这一分级机制，系统避免了对轻微偏差的过度干扰，同时能够在关键时间冲突出现时及时提示，从而在“提醒有效性”与“用户体验”之间取得平衡。

竞品对比与产品取舍

一、现有产品形态

当前主流旅行路线类产品，大致可以分为三种形态：

1. 规划工具型（偏流程驱动）

特点：

- 创建行程需填写城市 / 天数 / 名称
- 通过“添加行程内容”逐步构建计划
- 提供丰富模块（行李 / 账单 / 相册 / 备注等）

代表问题：

- 用户必须先“想清楚”，才能开始
- 上手成本高，探索成本大

本质：以“计划填写”为核心，而不是“空间探索”。

2. 地图聚合型（偏信息展示）

特点：

- 地图上聚合大量景点信息
- 支持收藏 → 再进入个人地图
- POI 信息密度高

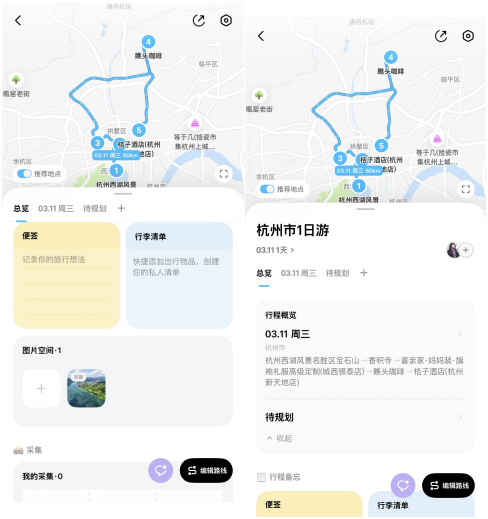
代表问题：

- 信息过载，空间结构混乱
- 操作路径长（收藏 → 我的地图）
- 无法即时形成路线

本质：以“信息展示”为核心，而不是“路径生成”。

3. 路线规划型（接近本产品）

特点：



- 支持多点构建行程
- 可生成基础路线
- 支持交通方式调整

代表问题：

- 默认入口为行程列表，而非地图
- 无法在地图上自由选点（依赖平台点位）
- 引入大量非核心模块（相册 / 备注 / 清单等）
- 行程展示以文本为主，缺乏时间结构表达

本质：路径能力存在，但“用户控制权”不足，产品仍然以平台结构为主导

二、核心问题总结

通过对比可以发现，这些产品的主要问题并不是“功能缺失”，而在于产品结构层面的偏差：

1. 入口设计问题：以列表或流程为起点，而非空间认知

   用户进入产品时，无法建立空间认知

2. 路径控制问题：依赖点位系统

   用户无法真正主导路径结构

3. 功能泛化问题：工具逐渐被内容和与记录功能侵蚀

   大量模块（备注 / 相册 / 清单）干扰核心体验

4. 表达方式问题：缺乏对事件与节奏的结构化表达

   只给“行程结果”，不展示“时间结构与节奏”

三、我的产品取舍

基于上述问题，我在 RouteGo 中做了几项明确选择：

1. 地图优先（空间先于列表）

   进入产品即进入地图，先建立空间认知

2. 自由选点（路径由用户构建）

   支持在地图任意位置添加节点，而非依赖预设点位

3. 内容化（保持工具纯度）

   不做相册、备注、清单等功能，专注路径与时间

4. 结构表达（从结果到过程）

   通过时间轴展示行程节奏，而非仅文本描述

四、核心定位

RouteGo 不是“行程记录工具”，而是“行程结构设计工具”，帮助用户从“填写计划”到“在地图上构建路径”，从“查看结果”到“理解过程”。

核心问题探索丨“附近逛逛”为什么难做

一、为什么需要附近逛逛

在完成路线编辑与时间系统之后，我开始思考一个新的问题：

在很多真实场景下，用户并不是“要规划一整天的行程”，而是只拥有一小段零散时间，却同样希望能够利用起来。这就是“附近逛逛”功能的出发点。

例如：

- 用户刚到一个城市，在前往酒店或景区之前，有几十分钟的空闲时间；
- 在高铁站或机场附近等待出发，希望在周边简单走一走；
- 与朋友约见之前，有一段不确定长度的空档时间；
- 在陌生城市或偏僻度假地点，想随便散步或兜风，但不知道应该往哪里走；

这些场景的共同特点是：

- 时间短
- 目标不明确
- 对环境缺乏认知

在这种情况下，用户的问题并不是“如何规划路线”，而是更基础的一步：

我现在可以去哪里走一走？

二、问题拆解丨从“找点”到“找路径”

我将“附近逛逛”定义为一个典型的碎片时间场景下的探索需求。但在进一步分析时，我发现，“附近逛逛”本身其实是一个非常模糊的用户表达，它并不对应一个单一的产品能力。

用户说“我想在附近逛一逛”，并不等于他在说同一类需求。我对这一需求进行了进一步拆解，将其还原为三种不同的问题类型：

- 找点（POI 探索）

用户希望知道附近有哪些可以去的地方，例如景点、商圈或餐饮点位。本质是“信息检索问题”。

- 找路径（路线生成）

用户并不关心具体有哪些点，而是希望得到一条可以直接去走的路线，例如“随便散步一圈”或“骑车逛一逛”。本质是“路径结构生成问题”。

- 做推荐（完整方案）

用户希望获得一套更完整的游玩安排，例如“半日游路线”。本质是“内容组织问题”。

最终，我对“附近逛逛”的定位进行了收敛：

- 找点 → 对应地图搜索与附近看点能力
- 做推荐 → 对应攻略类产品方向
- 附近逛逛 → 专注于路径生成能力

三、方案验证丨从“点到路径”的尝试与否定

在将“附近逛逛”收敛为**路径生成问题**之后，我最初的实现思路，其实是从“点”出发的。

当时我的一个直觉是：如果用户是在“逛”，那他更可能希望经过一些具有体验价值的节点，例如公园、河流、绿地或景观区域，而不是随机地在普通街道中移动。因此，我最初的思路是：优先筛选具有“可逛性”的地点，再将这些点连接成一条路径。

具体做法是：围绕当前位置搜索一批 POI，并优先选择具有景观属性的点位，例如公园、水系或开放空间；如果这类点位不足，则再退而选择其他类型节点；在得到一组候选点之后，再根据空间位置进行排序与拼接，生成一条候选路径，并通过路径服务逐段验证其可行性。

但在实际验证过程中，我很快发现，这一思路存在明显问题。具体表现为：

- 路线生成成功率较低，很多情况下无法得到一条完整可用的路线；
- 即使能够生成路线，整体耗时也较长，需要多次尝试与等待；
- 最终生成的路线质量不稳定，容易出现歪斜、绕行或不自然的路径结构；
- 需要通过大量路径验证来筛选结果，带来了较高的成本

也就是说，这一方案不仅“不优”，甚至“在实际使用中难以成立”

在这一过程中，我也逐渐意识到一个更本质的问题：用户在“逛”的过程中，真正感知的是路径，而不是点本身。即使筛选出的节点本身具有“可逛性”，也无法保证它们在空间上能够自然连接，更无法保证最终形成的路径是顺畅、合理的。这种“先找点、再拼路径”的思路，从一开始就在试图用局部信息去反推出整体结构，这使得路径始终处于不稳定、不可控的状态。

这一结论，使我开始从“如何选点”转向：**路径本身应该如何被定义与生成。**

四、认知升级丨从功能尝试到地图层思考

在前一轮验证之后，我意识到，“附近逛逛”的核心不是点，而是路径。点只是路径上的附着物，甚至很多时候，用户并不在乎经过了哪几个具体 POI，而更在乎整条路线给人的感觉是否顺畅、是否像一条真实会去走的路线。

也正是在这里，我开始重新定义“好路线”的标准。它不一定必须是一个几何意义上的完美闭环，但至少应该满足几个条件：主线清晰，不频繁折返；整体方向自然，不出现大量无意义分叉；能够回到起点附近，形成一个完整的小循环或近似循环；同时在时间与距离上，也要符合用户对“散步、骑行、兜风”这些不同场景的直觉预期。我后来判断路线好不好，已经不再是先看它有没有成环，而是先看它是不是“像一条人真的会去走的路线”。

但是继续往下做时，我又发现，这还不只是路径问题，它同时**受制于地图层本身的能力边界**。

例如，我最开始默认以“地图中心”作为附近逛逛的起点，因为这看起来最自

然：用户看到哪里，就从哪里开始逛。但后面我很快发现，地图中心并不一定等于一个真实可出发的位置。地图中心可能落在小区内部、草地、水边、景区空地，甚至是没有真实道路连接的位置。如果直接把它当成起点，那么后续路径生成就会天然不稳定，频繁出现“无路线”的情况。

这让我第一次真正意识到：**附近逛逛依赖的不是“地图中心”，而是“可出发位置”。**也就是说，在这个功能里，最底层的问题之一是地图层能不能先给我一个合理的起点。

我因此提出新的要求：系统不能直接假设当前中心点可用，而**应该先围绕这个中心去寻找一个更合理、更可通行的起始位置，再围绕这个位置生成候选路径**。这本质上已经不是简单的前端交互问题，而是开始进入“位置语义”层面的思考。

在进一步尝试过程中，我还遇到一个非常难以理解的问题：在视觉上，很多城市街区是规则且方正的，看起来完全可以围绕街区生成一条简单的环形路线。但在实际尝试中，即使是这种“肉眼看起来非常容易构建路径”的区域，系统依然很难稳定生成一条可用路线。例如，我曾尝试先人为构造一个“方形路径骨架”，再交由地图服务进行路径验证，但结果依然经常失败，或者生成的路径与预期偏差很大。

一开始我以为这是由于单行道或不可通行区域导致的，但在切换为步行模式后，这一问题依然存在，这说明问题并不只是通行限制。进一步分析后我逐渐意识到：**问题并不在于路径本身不可行，而在于地图服务并不理解“整体路径结构”**。

在当前的能力下，路径计算是以“点对点”为单位进行的，系统只会为每一段路径寻找局部最优解，而不会理解“这是一条需要保持形状和结构的整体路线”。也就是说，即使从几何上看，一条路径是规则且可行的，在实际计算中也可能被拆解为多个局部路径，从而偏离原本的整体结构。路径生成的问题，本质上不是几何问题，而是“局部路径计算”与“整体结构需求”之间的冲突。

再往后，我还进一步意识到，如果我始终完全依赖实时地图搜索与实时路径服务，那么这个功能会一直被两个问题限制住：第一，结果质量不可控；第二，调用成本会持续升高。因为只要路线生成逻辑还建立在“边搜索、边拼接、边逐

段验证”的模式上，系统就天然是在高成本地反复试错，而不是稳定地产生结果。

这一步让我开始重新思考**地图层和产品层之间的关系**。过去我更像是在“调用一个地图服务做功能”，但做到这里我才开始意识到，某些能力如果完全依赖外部地图接口，就永远只能停留在“拼接”和“试错”层面，而很难真正建立自己的生成能力。也正因为如此，我后来逐渐形成了一个更清晰的判断：

附近逛逛不应该在一开始就高度依赖真实路径验证，而**应该优先生成一条结构上合理的候选骨架，再把真实路径服务作为后置验证手段**。只有这样，系统才可能从“高成本试错”转向“低成本生成 + 少量验证”。

这一认知的变化，对我来说非常重要。因为它意味着我开始把问题从“怎么把这个功能做出来”，转向“这个功能到底依赖哪些底层能力、问题真正卡在哪一层”。到这个阶段，我已经不再把“附近逛逛”简单理解为一个新入口或一个推荐模块，而是把它看成一个牵涉位置、路径、地图语义、调用成本和产品边界的综合问题。

最终，我对这个功能形成了几个更明确的判断：

第一，它的核心不是推荐点，而是生成路径；
第二，它不是单纯的产品交互问题，而是会逼着系统重新面对地图层的能力边界；
第三，它的难点不在于某一段代码没写出来，而在于路径自然度、结果稳定性与外部调用成本之间始终存在张力；
第四，如果未来继续推进，一个更合理的方向应该是：路径骨架优先，POI 作为补充，真实路径验证后置化。

对我来说，附近逛逛这个功能仍然没有做好十一个遗憾，但是这部分最大的价值，是我第一次把一个复杂问题，从功能层一路拆到了结构层、地图层和系统层。这种思考方式，也让我开始意识到：很多看起来只是“加一个功能”的需求，真正难的地方往往根本不在界面上，而在更底层的能力组织方式上。

用户反馈与产品迭代

在项目开发过程后，我将当前版本部署为线上 Demo，并邀请朋友在真实使用场景中进行体验测试。

通过这一轮真实使用反馈，我发现了一些在开发过程中难以察觉的问题，这些问题主要集中在路径结构、地图表达以及移动端交互三个方面。

1. 行程结构问题：系统对“点数量”存在隐性限制

在用户使用过程中发现，当行程点数量少于 2 时（例如只有一个点或清空行程）时，系统会出现异常或无法正常表达当前状态。

问题本质：原有逻辑默认“行程至少包含两个点”，导致结构不完整

解决方式：重新设计路径触发逻辑，使系统支持：

0 点：无路径展示

1 点：仅展示位置

≥2 点：生成完整路径

2. 地图表达问题：多路线信息相互干扰

在测试过程中，当同时存在用户自定义路线与推荐路线时，地图显示出现混乱，用户难以区分主次关系。

问题本质：不同类型路线共用同一渲染层，缺乏层级区分

解决方式：对路线进行分层处理。用户路线用主线，推荐路线用参考线，并通过颜色与样式区分，使两者可以同时存在且互不干扰

3. 移动端交互问题：操作被地图层拦截

在移动端使用时，用户在地图选点后，出现“按钮无法点击”的问题

问题本质：地图层优先级过高，拦截了用户操作

解决方式：优化触控事件处理机制，使按钮交互优先响应

4. 表达优化：文案与产品定位不清晰

用户在使用过程中，对部分功能命名产生理解偏差

优化内容：

“一键规划路线” → “一键优化路线”

页面标题统一为“走起 - 旅行路线设计工具”

通过这一轮用户反馈，我逐渐意识到：

- 产品问题往往不是出现在“功能缺失”，而是出现在结构细节与使用路径中
- 很多问题只有在真实使用场景下才会暴露，而不是在开发过程中被发现
- 产品的稳定性，来自于结构设计、表达方式与交互逻辑的共同作用

技术问题与解决

问题一 ：拖拽排序中锁定节点被覆盖

一、问题背景

在实现行程列表拖拽排序功能时，我引入了“节点锁定”能力，用于保证起点或关键节点位置不被改变。初始设计中，我限制了被锁定节点本身不可拖动。在实际测试中发现，虽然锁定节点无法被直接拖动，但其他节点仍然可以拖动到其位置，从而间接改变了锁定节点的顺序，导致锁定机制失效。

二、问题定位

最初我只关注了“拖拽源”的限制，即谁可以被拖动，但忽略了“目标位置”的约束。本质上，问题不是“能不能拖”，而是“能不能占这个位置”。

三、解决方案

我将约束从“拖拽行为”扩展到“结果结构”，在拖拽结束阶段引入合法位置判断，通过限制目标索引范围，使锁定节点的位置无法被其他节点占用。

四、关键收获

拖拽约束不能只作用于操作过程，而必须作用于最终数据结构。

问题 2：地图加点与拖动操作冲突

一、问题背景

在实现“地图点击加点”功能时，用户可以通过点击地图任意位置，将该位置加入行程节点。同时，地图本身也支持拖动操作用于调整视野。在实际使用过程中，这两个交互发生了冲突：用户在尝试点击加点时，系统往往将其识别为地图拖动，导致无法成功添加节点，影响基础操作体验。

二、问题定位

最初我尝试通过拦截点击行为来区分“点击”和“拖动”，但实际效果并不稳定。进一步分析后发现，这类问题的核心不在于“是否触发点击”，而在于系统在更早的阶段已经将用户操作判定为“拖动”。也就是说，当用户手指或鼠标产生轻微移动时，系统优先认为用户是在拖动地图，而不是点击某一点。本质上，这不是简单的事件冲突，而是：用户操作意图（点击 vs 拖动）的识别问题

三、解决方案

为了解决这一问题，我没有再依赖单一的点击判断，而是改为从“用户行为”出发进行识别。具体来说，我在用户操作的起点和终点之间引入了“位移判断”：当用户按下并释放的过程中，如果位置变化非常小，则认为是一次点击；如果产生了明显位移，则判定为拖动行为。通过这种方式，将交互判断从“是否触发点击”升级为“用户是否发生移动”，从而稳定地区分两种操作。

四、关键收获

交互冲突问题，本质往往不是“事件处理错误”，而是“用户意图判断不准确”

相比依赖单一触发条件，更可靠的方式是基于完整操作过程进行判断

问题 3：路径计算异常与结果不稳定问题

一、问题背景

在实现多点路线规划时，系统需要根据用户选择的交通方式（步行 / 骑行 / 驾车），调用地图服务生成每一段路径，并将其拼接为完整路线。

- 在初期实现中，路径结果出现了多种异常情况，例如：
- 路线出现不合理的折返或绕行
- 某些路段无法正常绘制，显示为简化的虚线路径
- 同一条路线在多次刷新后结果不一致

二、问题定位

在排查过程中，我逐步发现问题并非单一原因，而是由多个因素叠加导致：

一方面，不同交通方式返回的路径数据结构存在差异，如果使用统一方式解析，容易导致路径解析错误，从而出现异常路线。另一方面，在多段路径连续请求的情况下，如果请求尚未完成就进入下一步处理，系统会误认为路径获取失败，从而触发兜底逻辑（即使用简化路径代替），表现为虚线。此外，频繁重复调用路径接口，也会导致部分请求失败或结果不稳定。

本质上，这一问题并不是“路径算错了”，而是：路径数据解析不统一 + 异步请求时序不稳定 + 调用策略不合理

三、解决方案

针对上述问题，我从三个层面进行了优化：

首先，在路径解析层，对不同交通方式的数据结构进行区分处理，确保每种路径都按照其真实结构进行解析，避免错误拼接。

其次，在请求流程上，改为严格控制路径请求的执行顺序，确保每一段路径

都在成功返回后再进入下一步处理，从而避免因请求未完成而误判失败。

最后，在调用策略上，引入路径缓存机制：对于相同起点、终点和交通方式的路径结果进行缓存，避免重复请求，不仅提升了性能，也显著提高了结果稳定性。

四、关键收获

- 路径问题本质上不是“算法问题”，而是“数据结构理解 + 请求流程控制”的问题
- 在依赖第三方服务时，稳定性往往来自于“控制调用方式”，而不是“依赖接口本身”
- 从“能跑通”到“稳定可用”，关键在于处理边界情况与异常路径

问题 4：地点名称更新后，按钮点击失效

一、问题背景

在实现“地图点击加点”功能时，我希望用户点击地图后，系统先生成一个临时标记，再通过逆地理编码自动补全地点名称。也就是说，用户第一次看到的可能是“查询中”这样的临时状态，随后会被真实地点名称替换。这个流程本身没有问题，但在实际使用中我发现：地点名称更新之后，原本可以点击的“加入行程”按钮会突然失效。

二、问题定位

最初我以为是按钮本身的点击逻辑有问题，但进一步排查后发现，问题并不在“按钮功能”本身，而在于“按钮所在的界面元素已经不是原来的那个了”。

当系统根据逆地理编码结果更新 marker 内容时，实际上并不是在原有内容上局部修改，而是重新生成了一整块新的界面内容。这样一来，原本绑定在旧按钮上的交互逻辑也会一起消失。

本质上，这个问题不是“按钮失效”，而是：内容更新带来了界面重建，导致原有交互状态被清空

三、解决方案

解决这个问题时，我没有继续依赖“旧按钮”的状态，而是把事件绑定逻辑从内容本身中拆出来，变成一个可以重复执行的独立步骤。也就是说，每当 marker 内容被更新后，我都会重新对新的按钮进行一次事件绑定，确保用户看到的当前按钮一定具备正确的交互能力。通过这种方式，我把原本隐含在旧界面上的交互关系，改成了“内容更新一次，交互重新绑定一次”的显式管理逻辑，从而彻底解决了名称更新后按钮失效的问题。

四、关键收获

- 在动态渲染场景下，界面更新不只是“文字变化”，有时意味着整块内容被重新生成
- 一旦界面被重建，原有交互不能默认继续生效，必须显式重新绑定
- 前端问题很多时候不是“功能没写对”，而是“错误地依赖了旧界面状态。

问题 5：搜索输入过程中被打断（中文输入问题）

一、问题背景

在实现“搜索地点加入行程”功能时，我为提升响应速度，设计为用户输入时实时触发搜索，并动态更新搜索结果列表。在英文输入场景下，这一逻辑运行正常。但在中文输入过程中，出现了明显问题：用户在输入拼音时，输入内容会被中断，候选词未完成选择就被打断，甚至出现光标跳动或输入框被清空的情况。

二、问题定位

最初我以为是输入框本身的问题，但进一步排查后发现，问题并不在输入组件，而在于“搜索结果更新触发了界面重新渲染”。

当用户输入时，系统会实时更新搜索结果，而每次更新都会导致相关区域重新渲染。在这个过程中，输入框的状态（包括正在输入的拼音、候选词选择等）被打断，从而导致输入异常。

本质上，这个问题不是“输入功能出错”，而是：渲染更新打断了用户的输入过程

三、解决方案

为了解决这一问题，我对搜索触发机制进行了调整，不再在每一次输入变化时立即触发渲染，而是对输入状态进行控制。

具体来说，在用户处于连续输入（尤其是中文拼音输入）过程中时，系统会暂缓触发搜索更新，仅在输入稳定或完成后再执行渲染，从而避免在输入过程中打断用户操作。我减少了不必要的界面刷新范围，确保输入框本身在更新过程中保持稳定，不被重新生成或重置。

通过这种方式，实现了“输入优先、渲染延后”的处理逻辑，使搜索体验恢复正常。

四、关键收获

- 输入问题本质不是“输入框问题”，而是“渲染时机问题”
- 在涉及用户输入的场景中，系统必须优先保证输入连续性，而不是追求实时更新
- 动态更新能力需要与用户操作节奏进行协调，否则会破坏基本体验

问题 6：路径请求成本过高与结果不稳定

一、问题背景

在实现多点路线规划后，系统需要为每一段路径分别请求地图服务生成路线。例如，一条包含多个地点的行程，会被拆分为多段路径分别计算。

在实际使用过程中，我发现两个明显问题：一是路径请求次数随着节点数量增加迅速增长，导致加载时间变长；二是同一条路线在重复计算时，结果偶尔会出现差异，影响整体稳定性。此外，频繁调用接口也带来了明显的成本压力。

二、问题定位

进一步分析后发现，这些问题的根源在于路径请求是“重复且无记忆”的：每一次重新计算路线时，系统都会对所有路径重新发起请求，即使起点、终点和交通方式完全相同，也不会复用已有结果。这不仅增加了请求次数，也使系统完全依赖外部服务的实时响应，一旦某次请求失败或波动，就会影响最终结果。

本质上，这个问题不是“请求太多”，而是：系统缺乏对已有路径结果的复用能力

三、解决方案

为了解决这一问题，我引入了路径缓存机制，将每一段路径结果进行存储与复用。

具体做法是，将“起点 + 终点 + 交通方式”作为唯一标识，当系统需要计算路径时，优先检查是否已有对应结果，如果存在则直接复用，而不再发起新的请求。同时，对于请求失败或不可达的路径组合，也进行记录，避免重复尝试无效请求。

通过这一机制，系统从“每次重新计算”转变为“尽可能复用已有结果”，显著减少了接口调用次数，并提升了整体响应速度与稳定性。

四、关键收获

- 性能问题本质往往不是“速度不够”，而是“重复计算过多”
- 在依赖第三方服务时，必须主动构建“缓存与复用机制”，而不能完全依赖接口稳定性
- 优化不仅是技术问题，也直接关系到成本控制与用户体验

项目总结与思考

在 RouteGo 的设计与实现过程中，我逐渐发现，这个项目的难点并不在于“功能实现”，而在于如何将一个看似简单的需求，拆解为可被构建的结构问题。

一开始，我关注的是“如何规划路线”，但在不断尝试与迭代中，我逐渐意识到：用户真正需要的，并不是一个“路线结果”，而是一种对行程的整体掌控能力。

这使我从“功能设计”转向“结构设计”，并围绕三个核心维度重新理解这个问题：

- 空间：用户需要理解“去哪里”
- 路径：用户需要控制“怎么走”
- 时间：用户需要掌控“何时发生”

在这一过程中，我也逐步形成了一种更清晰的产品认知：

1. 从“结果导向”到“过程导向”
   传统产品更倾向于直接给出结果，而我更关注如何让用户理解结果的形成过程，使其具备调整与决策能力
2. 从“功能堆叠”到“结构收敛”
   在面对复杂需求时，并不是不断增加功能，而是通过拆解与取舍，将问题收敛为少数关键能力
3. 从“调用能力”到“理解边界”
   在实现过程中，我逐渐意识到产品不仅依赖于已有技术能力，还需要理解其边界，并在此基础上重新设计解决方案

最终，我将 RouteGo 定义为一个帮助用户构建与理解行程结构的工具。

帮助用户掌控自由行程中的节奏感、空间感与时间感